

AI-Driven Automated Interviewer for Project Presentations: Comprehensive Project Specification

1. Executive Summary

The "AI-Driven Automated Interviewer for Project Presentations" is an advanced, multimodal artificial intelligence system designed to autonomously evaluate, question, and score students or professionals during technical project demonstrations. By leveraging State-of-the-Art (SOTA) Large Language Models (LLMs), real-time Speech-to-Text (STT) transcription, and dynamic prompt engineering, this system acts as a highly capable virtual panelist. It actively listens to the presentation, formulates context-aware questions on the fly, and generates a structured, unbiased evaluation report based on strict, predefined academic or corporate rubrics.

2. Problem Statement & Motivation

Traditional project evaluations and technical interviews are heavily bottlenecked by human constraints. Evaluators are subject to unconscious biases, cognitive fatigue, scheduling conflicts, and varying levels of domain expertise. Furthermore, manual grading lacks the scalability required for large university cohorts or mass corporate hiring initiatives. An AI-driven approach standardizes the baseline of evaluation, ensures that probing questions are directly aligned with the presenter's unique content, and provides instantaneous, highly detailed feedback without scaling costs.

3. Core System Architecture

The system relies on a modular pipeline architecture to ensure low latency and high accuracy during live interactions.

- **Perception Module (Audio/Visual Input):** Captures presentation audio using tools like OpenAI's Whisper for high-accuracy, low-latency transcription. Optical Character Recognition (OCR) via tools like Tesseract can optionally extract code and text directly from the presenter's screen.
- **Cognitive Engine (LLM Orchestration):** Serves as the core reasoning engine. It

maintains a rolling context window of the presentation transcript. Frameworks like LangChain or LlamaIndex manage the memory state and feed the context to a local inference engine (e.g., Ollama running Llama 3) or a cloud API.

- **Interrogator Module (Dynamic Q&A):** Triggers at specific intervals or at the conclusion of the pitch. The LLM generates probing questions challenging the presenter's architectural choices, technical depth, and originality. Text-to-Speech (TTS) can be utilized to vocalize these questions naturally.
- **Evaluator & Reporting Module:** Parses the final transcript—including the critical Q&A session—to output a strictly formatted JSON report scoring the candidate against specific parameters.

4. System Workflow & Data Pipeline

1. **Initialization:** The session begins. The system initiates continuous audio recording and screen-text sampling (via tools like mss).
2. **Real-time Transcription:** Audio is chunked and transcribed. Text context is appended to an ongoing session memory array.
3. **Q&A Generation:** Upon detecting the conclusion of the presentation, the LLM analyzes the session memory and synthesizes 2 to 4 domain-specific questions to test the presenter's boundaries.
4. **Response Capture:** The presenter answers the questions verbally. These responses are transcribed and appended to the final context, capturing how well they defend their implementation.
5. **Evaluation Generation:** A final, highly restrictive prompt containing a JSON schema is sent to the LLM (with temperature set to 0.0) to generate the deterministic final scores.

5. Evaluation Metrics & Scoring Rubric

The AI evaluator assesses candidates across four primary dimensions, generating a quantitative score (0.0 to 10.0) alongside qualitative feedback for each:

- **Technical Depth:** Assesses the complexity of the tech stack, architectural decisions, and the presenter's ability to articulate the underlying engineering principles of their solution.
- **Clarity of Explanation:** Evaluates the structure of the presentation, the coherence of the narrative, and the presenter's ability to distill complex technical topics into understandable concepts.
- **Originality:** Measures the novelty of the solution, the uniqueness of the approach, and the absence of purely boilerplate or tutorial-driven implementation.
- **Understanding of Implementation:** Determines the authenticity of the candidate's work

based on how well they handle dynamic Q&A probing, identifying if they truly understand the code they are presenting.

6. Technology Stack Formulation

System Component	Recommended Technology	Purpose & Justification
STT / Audio Processing	Whisper (OpenAI) / pyaudio	Real-time, highly accurate voice-to-text conversion robust against various accents.
Screen / Context Capture	mss / Tesseract OCR	Extracting contextual text (like code snippets) from slides or IDEs during the presentation.
LLM Reasoning Engine	Ollama (Llama 3 8B)	Natural language comprehension, Q&A synthesis, and rubric-based grading.
Orchestration Framework	Python 3.x / Threading	Connecting transcripts to the LLM, managing session memory, and parsing JSON outputs.

7. JSON Schema Enforcement Configuration

To ensure the LLM outputs a predictable, machine-readable format for downstream dashboards or databases, strict schema enforcement is required at the API level.

8. Ethical Considerations & Bias Mitigation

While the AI interviewer provides exceptional standardization, strict guardrails must be maintained to ensure fairness:

- **Acoustic and Dialect Bias:** The STT engine must be thoroughly benchmarked against diverse global accents to ensure non-native speakers are not penalized by poor transcriptions.
- **Hallucination Control:** The LLM's temperature parameter must be configured to 0.0 during the final evaluation phase to strictly anchor the AI's grading logic to the transcript, preventing it from inventing presentation details.

9. Appendix A: Reference Implementation (Python)

Below is the core orchestration script that ties the STT, OCR, and local LLM (Ollama) together using background threading.

```
import threading
import time
import queue
import pytsx3
import pyaudio
import wave
import os
import ollama
import pytesseract
from mss import mss
from PIL import Image
from faster_whisper import WhisperModel

# =====
# 0. System Configurations (Mac Specific)
# =====
pytesseract.pytesseract.tesseract_cmd = r'/opt/homebrew/bin/tesseract'

# =====
# 1. Application State & Settings
# =====
LLM_MODEL_NAME = "llama3:8b"
CHUNK_DURATION = 5
presentation_context = []
audio_queue = queue.Queue()

# =====
```

```
# 2. AI Model Initialization
```

```
# =====
```

```
tts_engine = pyttsx3.init()
```

```
tts_engine.setProperty('rate', 160)
```

```
def speak(text):
```

```
    print(f"\n[Interviewer]: {text}")
```

```
    tts_engine.say(text)
```

```
    tts_engine.runAndWait()
```

```
whisper_model = WhisperModel("tiny.en", device="cpu", compute_type="int8")
```

```
# =====
```

```
# 3. Screen Capture & OCR Thread
```

```
# =====
```

```
def extract_screen_text():
```

```
    with mss() as sct:
```

```
        monitor = sct.monitors[1] if len(sct.monitors) > 1 else sct.monitors[0]
```

```
        while True:
```

```
            screenshot = sct.grab(monitor)
```

```
            img = Image.frombytes("RGB", screenshot.size, screenshot.bgra, "raw", "BGRX")
```

```
            text = pytesseract.image_to_string(img).strip()
```

```
            if len(text) > 20:
```

```
                presentation_context.append(f"[SCREEN OCR]: {text}")
```

```
            time.sleep(10)
```

```
# =====
```

```
# 4. Audio Capture Thread
```

```
# =====
```

```
def record_audio():
```

```
    FORMAT = pyaudio.paInt16
```

```
    CHANNELS = 1
```

```
    RATE = 16000
```

```
    CHUNK = 1024
```

```
    audio = pyaudio.PyAudio()
```

```
    stream = audio.open(format=FORMAT, channels=CHANNELS, rate=RATE, input=True,  
frames_per_buffer=CHUNK)
```

```
    while True:
```

```
        frames = []
```

```
        for _ in range(0, int(RATE / CHUNK * CHUNK_DURATION)):
```

```
            data = stream.read(CHUNK)
```

```
            frames.append(data)
```

```
    temp_file = f"temp_{threading.get_ident()}.wav"
```

```

    wf = wave.open(temp_file, 'wb')
    wf.setnchannels(CHANNELS)
    wf.setsampwidth(audio.get_sample_size(FORMAT))
    wf.setframerate(RATE)
    wf.writeframes(b"".join(frames))
    wf.close()

    audio_queue.put(temp_file)

# =====
# 5. Audio Transcription Thread
# =====
def transcribe_audio():
    while True:
        file_path = audio_queue.get()
        segments, _ = whisper_model.transcribe(file_path, beam_size=5)
        text = "".join([segment.text for segment in segments]).strip()

        if text:
            presentation_context.append(f"[STUDENT SPOKE]: {text}")
            print(f"\n[Transcribed]: {text}")

        if os.path.exists(file_path):
            os.remove(file_path)

# =====
# 6. AI Interviewer Logic (Ollama)
# =====
def generate_question():
    while True:
        time.sleep(45)

        if len(presentation_context) < 3:
            continue

        context_str = "\n".join(presentation_context[-15:])
        prompt = f"""You are a software engineering technical interviewer. Based on the student's recent
presentation code and speech, ask ONE short, specific follow-up question. Do not praise them. Be
direct.\n\nContext:\n{context_str}"""

        try:
            response = ollama.generate(
                model=LLM_MODEL_NAME,
                prompt=prompt,
                options={"num_ctx": 8192, "temperature": 0.7}
            )
            question = response['response'].strip()
            speak(question)

```

```

    except Exception as e:
        print(f"\n[Error generating question]: {e}")

def evaluate_performance():
    total_logs = len(presentation_context)
    if total_logs == 0:
        print("\n[CRITICAL WARNING]: No data was captured during this session!")
    return

    safe_context = presentation_context[-40:] if total_logs > 40 else presentation_context
    context_str = "\n".join(safe_context)

    prompt = f"Review the following transcript and evaluate the student's performance out of
10.\n\nTranscript:\n{context_str}"

    evaluation_schema = {
        "type": "object",
        "properties": {
            "technical_depth": { "type": "number" },
            "clarity_of_explanation": { "type": "number" },
            "originality": { "type": "number" },
            "understanding_of_implementation": { "type": "number" },
            "feedback_summary": { "type": "string" }
        },
        "required": ["technical_depth", "clarity_of_explanation", "originality", "understanding_of_implementation",
"feedback_summary"]
    }

    try:
        response = ollama.generate(
            model=LLM_MODEL_NAME,
            prompt=prompt,
            format=evaluation_schema,
            options={"num_ctx": 8192, "temperature": 0.0}
        )
        print(response['response'].strip())
    except Exception as e:
        print(f"\n[CRITICAL ERROR] Failed to communicate with Ollama: {e}")

# =====
# 7. Main Execution Orchestrator
# =====
if __name__ == "__main__":
    input("Press Enter to start the presentation...")
    threading.Thread(target=extract_screen_text, daemon=True).start()
    threading.Thread(target=record_audio, daemon=True).start()
    threading.Thread(target=transcribe_audio, daemon=True).start()
    threading.Thread(target=generate_question, daemon=True).start()

```

```
speak("I am ready. Please begin your presentation.")
```

```
try:
```

```
    while True:
```

```
        time.sleep(1)
```

```
except KeyboardInterrupt:
```

```
    print("\n\nEnding Interview and compiling data...")
```

```
    evaluate_performance()
```